数值线性代数　第 29 讲：移位 QR 算法

Lloyd N. Trefethen、David Bau III　中文阅读版

让 QR 迭代真正“飞起来”的关键，是在每一步引入移位 $A \mapsto A - \mu I$。本讲解释这一思想如何通过与 Rayleigh 商迭代的隐含联系带来三次收敛。

与逆迭代的联系

继续假设 $A \in \mathbb{R}^{m\times m}$ 是实对称矩阵，特征值为实数 $\{\lambda_j\}$，特征向量 $\{q_j\}$ 标准正交。

我们已经看到，“纯”QR 算法（算法 28.1）等价于把同时迭代应用于单位矩阵；特别地，结果的第一列按照从 e_1 出发的幂迭代演化。这个观察有一个对偶版本：算法 28.1 也等价于把同时逆迭代应用于一个“翻转后的”单位矩阵 P，因此结果的第 m 列按照从 e_m 出发的逆迭代演化。

下面建立这个结论。为避免原书的下划线记号在纯文本中产生歧义，本文用 $Q^{(j)}$ 表示第 j 步 QR 分解得到的正交因子，并用

$$\mathcal{Q}^{(k)} = Q^{(1)}Q^{(2)}\cdots Q^{(k)}$$

表示累计的正交因子；相应的累计上三角因子记为 $\mathcal{R}^{(k)}$。上一讲的式 (28.14) 说明

$$\mathcal{Q}^{(k)} = \prod_{j=1}^{k} Q^{(j)} = \begin{bmatrix} q_1^{(k)} & q_2^{(k)} & \cdots & q_m^{(k)} \end{bmatrix},$$

它正是同时迭代在第 k 步（式 (28.9)）出现的正交矩阵。换一种说法，$\mathcal{Q}^{(k)}$ 是 QR 分解

$$A^k = \mathcal{Q}^{(k)}\mathcal{R}^{(k)}$$

中的正交因子。

现在把这个公式求逆。由于 A^{-1} 仍然是对称矩阵，有

$$A^{-k} = \big(\mathcal{R}^{(k)}\big)^{-1}\big(\mathcal{Q}^{(k)}\big)^{T} = \mathcal{Q}^{(k)}\big(\mathcal{R}^{(k)}\big)^{-T}. \tag{29.1}$$

令 P 为把行或列的顺序完全翻转的 $m \times m$ 置换矩阵：

$$P = \begin{bmatrix} 0 & \cdots & 0 & 1 \\ \vdots & \ddots & 1 & 0 \\ 0 & 1 & \ddots & \vdots \\ 1 & 0 & \cdots & 0 \end{bmatrix}, \qquad P^2 = I.$$

于是 (29.1) 可以改写为

$$A^{-k}P = \big[\mathcal{Q}^{(k)}P\big]\big[P\big(\mathcal{R}^{(k)}\big)^{-T}P\big]. \tag{29.2}$$

乘积中的第一个因子 $\mathcal{Q}^{(k)}P$ 是正交矩阵。第二个因子 $P(\mathcal{R}^{(k)})^{-T}P$ 是上三角矩阵：先从下三角矩阵 $(\mathcal{R}^{(k)})^{-T}$ 出发，上下翻转一次，再左右翻转一次即可看到这一点。因此，(29.2) 可以解释为 $A^{-k}P$ 的 QR 分解。换句话说，我们实际上是在对 A^{-1} 做初始矩阵为 P 的同时迭代，也就是对 A 做同时逆迭代。特别地，$\mathcal{Q}^{(k)}P$ 的第一列——也就是 $\mathcal{Q}^{(k)}$ 的最后一列——正是对向量 e_m 做 k 步逆迭代所得的结果。

与移位逆迭代的联系

因此，QR 算法同时具有同时迭代和同时逆迭代两种解释；这两个视角完全对称。但是，正如第 27 讲所见，幂迭代和逆迭代之间有一个巨大差异：逆迭代可以通过移位任意加速。我们越能准确估计某个特征值 $\mu \approx \lambda_j$，用移位矩阵 $A - \mu I$ 做一步逆迭代时得到的效果就越好。算法 28.2 说明了如何在 QR 的一步迭代中引入移位；这恰好对应于同时迭代和同时逆迭代中的移位，因此它们的有利效果也完全相同。

令 $\mu^{(k)}$ 表示 QR 算法第 k 步选取的特征值估计。由算法 28.2，从第 $k-1$ 步到第 k 步的移位 QR 关系为

$$A^{(k-1)} - \mu^{(k)}I = Q^{(k)}R^{(k)}, \qquad A^{(k)} = R^{(k)}Q^{(k)} + \mu^{(k)}I.$$

这意味着

$$A^{(k)} = (Q^{(k)})^T A^{(k-1)} Q^{(k)}, \tag{29.3}$$

并且通过归纳得到

$$A^{(k)} = (\mathcal{Q}^{(k)})^T A \mathcal{Q}^{(k)},$$

这与式 (28.17) 没有变化。不过，式 (28.16) 不再成立。现在得到的分解是

$$\begin{aligned} (A - \mu^{(k)}I)(A - \mu^{(k-1)}I)\cdots(A - \mu^{(1)}I) \\ = \mathcal{Q}^{(k)}\mathcal{R}^{(k)}. \end{aligned} \tag{29.4}$$

这是同时迭代的一个移位版本(证明略)。换句话说，$\mathcal{Q}^{(k)} = \prod_{j=1}^{k} Q^{(j)}$ 是乘积 $\prod_{j=1}^{k}(A-\mu^{(j)}I)$ 的正交化结果。$\mathcal{Q}^{(k)}$ 的第一列是使用这些移位从 e_1 出发的移位幂迭代结果；最后一列则是使用同一组移位从 e_m 出发做 k 步移位逆迭代的结果。如果移位是足够好的特征值估计，最后一列就会快速收敛到一个特征向量。

与 Rayleigh 商迭代的联系

我们已经在移位 QR 算法中发现了一个强大的工具：移位逆迭代。要使 QR 算法最后一列快速收敛，还需要解决如何选择移位的问题。

Rayleigh 商是一个自然的起点。若要估计 $\mathcal{Q}^{(k)}$ 最后一列所近似的特征向量对应的特征值，最自然的做法就是对这一列使用 Rayleigh 商。这给出

$$\mu^{(k)} = \frac{(q_m^{(k)})^T A q_m^{(k)}}{(q_m^{(k)})^T q_m^{(k)}} = (q_m^{(k)})^T A q_m^{(k)}. \tag{29.5}$$

如果每一步都把这个数选作移位，那么特征值估计 $\mu^{(k)}$ 和特征向量估计 $q_m^{(k)}$ 就与从 e_m 出发执行 Rayleigh 商迭代所得到的结果完全相同。因此，QR 算法具有三次收敛性，具体说是 $q_m^{(k)}$ 三次收敛到一个特征向量。

注意，在 QR 算法中，$q_m^{(k)}$ 的 Rayleigh 商就是 $A^{(k)}$ 的 (m,m) 条目，因此这个移位几乎不需要额外计算。上一讲结尾已经提到过这一点，这里再明确推导一次。从 (29.3) 出发，有

$$A_{mm}^{(k)} = e_m^T A^{(k)} e_m = e_m^T (\mathcal{Q}^{(k)})^T A \mathcal{Q}^{(k)} e_m = (q_m^{(k)})^T A q_m^{(k)}. \tag{29.6}$$

因此，(29.5) 等价于简单地取 $\mu^{(k)} = A_{mm}^{(k)}$。这称为 Rayleigh 商移位。

Wilkinson 移位

Rayleigh 商移位在一般情形下给出三次收敛，但并不能保证所有初始条件都收敛。用一个简单例子即可看出这一点。考虑矩阵

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \tag{29.7}$$

无移位 QR 算法完全不收敛：

$$A = Q^{(1)} R^{(1)} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

而

$$A^{(1)} = R^{(1)}Q^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = A.$$

不过，Rayleigh 商移位 $\mu = A_{mm}$ 也没有作用，因为 $A_{mm} = 0$。因此在最坏情形下，带 Rayleigh 商移位的 QR 算法可能失败。

问题来自特征值的对称性：一个特征值为 $+1$，另一个为 -1。当我们试图改进特征值估计 0 时，算法偏向两个特征值的倾向完全相同，因而估计不会改进。我们需要一种能够打破这种对称性的特征值估计。下面给出一种选择。令 B 表示 $A^{(k)}$ 右下角的 2×2 子矩阵：

$$B = \begin{bmatrix} a_{m-1} & b_{m-1} \\ b_{m-1} & a_m \end{bmatrix}.$$

Wilkinson 移位定义为 B 中距离 a_m 更近的那个特征值；如果两个特征值距离相同，则任意选取其中一个。Wilkinson 移位的一个数值稳定公式是

$$\mu = a_m - \operatorname{sign}(\delta) \frac{b_{m-1}^2}{|\delta| + \sqrt{\delta^2 + b_{m-1}^2}}, \qquad \delta = \frac{a_{m-1} - a_m}{2}. \tag{29.8}$$

如果 $\delta = 0$，$\operatorname{sign}(\delta)$ 可以任意设为 1 或 -1。

和 Rayleigh 商移位一样，Wilkinson 移位在一般情形下达到三次收敛；此外可以证明，在最坏情形下它至少达到二次收敛。特别地，带 Wilkinson 移位的 QR 算法在精确算术中总会收敛。

在例 (29.7) 中，Wilkinson 移位为 $+1$ 或 -1。因此对称性被打破，算法一步就会收敛。

稳定性与精度

至此，我们完成了对 QR 算法机制的讨论；许多实际细节仍被省略，例如降阶条件以及移位的“隐式”实现策略。最后说明一下稳定性和精度。

QR 算法使用正交矩阵，因此它后向稳定并不令人意外。和前面各讲一样，最简单的表述方式是：令 $\widetilde{\Lambda}$ 表示浮点运算中计算得到的 A 的对角化，令 $\widetilde{Q}$ 表示所有数值计算得到的 Householder 反射（或 Givens 旋转）之积所对应的、严格正交的矩阵。可以证明如下结果。

定理 29.1. 设实对称三对角矩阵 $A \in \mathbb{R}^{m \times m}$ 在满足 (13.5)、(13.7) 的计算机上由 QR 算法（算法 28.2）对角化，并按上述方式定义 $\widetilde{\Lambda}$ 和 $\widetilde{Q}$。则存在某个 $\delta A \in \mathbb{C}^{m \times m}$，使得

$$\widetilde{Q}\,\widetilde{\Lambda}\,\widetilde{Q}^* = A + \delta A, \qquad \frac{\|\delta A\|}{\|A\|} = O(\varepsilon_{\text{machine}}). \tag{29.9}$$

和本书中的大多数算法一样，QR 算法给出了一个略有扰动的问题的精确解。结合定理 26.1 和定理 29.1，可以看到，三对角化之后再执行 QR 算法，是一个计算矩阵特征值的后向稳定算法。再把这个结论与式 (26.4) 关于实对称矩阵（正规矩阵的特殊情形）特征值扰动的结果结合起来，就得到计算特征值 $\widetilde{\lambda}_j$ 满足

$$\frac{|\widetilde{\lambda}_j - \lambda_j|}{\|A\|} = O(\varepsilon_{\text{machine}}). \tag{29.10}$$

对于一个只需要约 $\frac{4}{3}m^3$ 次浮点运算的算法来说，这个结果相当不错；它只相当于计算两个 $m \times m$ 矩阵乘积所需运算量的三分之二。

习题

29.1. 这道五部分习题要求只使用基本构件编写一个 MATLAB 程序，求出实对称矩阵的全部特征值。不必通过最优地利用对称性或零结构来追求最优常数；整个问题可以用大约 50 行程序解决。

(a) 编写函数 $T = \texttt{tridiag}(A)$，通过正交相似变换把实对称 $m \times m$ 矩阵约化为三对角形式。程序只能使用 MATLAB 的基本操作，不能使用内置函数 ‘hess‘ 等。除舍入误差外，输出矩阵 T 应当对称且为三对角矩阵；如果愿意，可以在最后加一行强制 T 精确对称且精确三对角。以 $A = \texttt{hilb}(4)$ 为例运行程序。

(b) 编写函数 $T_{\text{new}} = \texttt{qralg}(T)$，对实三对角矩阵 T 执行无移位 QR 算法。每一步的 QR 分解可以使用习题 10.2 中的程序 $[W, R] = \texttt{house}(A)$ 和 $Q = \texttt{formQ}(W)$（如果已经有），也可以使用 MATLAB 命令 ‘qr‘；为了提高效率，还可以编写使用 Givens 旋转或 2×2 Householder 反射的新程序，避免 $m \times m$ 运算。也可以在每一步强制对称性和三对角结构。当

$$|t_{m,m-1}| < 10^{-12}$$

时停止，并将当前三对角矩阵 T 作为 T_{new} 返回（这还远算不上工业级的收敛判据）。同样对 $A = \texttt{hilb}(4)$ 运行。

(c) 编写驱动程序：(i) 调用 ‘tridiag‘；(ii) 调用 ‘qralg‘ 得到一个特征值；(iii) 对更小的矩阵调用 ‘qralg‘ 得到另一个特征值；如此继续，直到得到 A 的全部特征值。设置程序，使其把每一次 QR 迭代中的 $|t_{m,m-1}|$ 存入一个向量，并在最后生成这些数值相对于 QR 分解次数的 ‘semilogy‘ 图。随着降阶进行，m 会从 ‘length(A)‘ 逐步变为 ‘length(A)-1‘，一直到 3，最后到 2，因此图形会呈现锯齿状。对 $A = \texttt{hilb}(4)$ 运行程序；输出应包括一组特征值和一幅“锯齿图”。

(d) 修改 ‘qralg‘，使其在每一步使用 Wilkinson 移位。对同一例子提交新的锯齿图。

(e) 对矩阵

$$A = \mathrm{diag}(15 : -1 : 1) + \mathrm{ones}(15, 15)$$

重新运行程序，并分别生成使用移位和不使用移位的两幅锯齿图。讨论在这个矩阵和前一个矩阵上观察到的收敛速率：它是线性的、超线性的、二次的、三次的，还是其他类型？谈论“每个特征值需要多少次 QR 迭代”有意义吗？